



MANUEL D'UTILISATION DU SIMULATEUR

Apprentissage par renforcement d'une politique de navigation mono-agent avec l'algorithme SAC

SUJET DU STAGE : Étude d'algorithmes d'apprentissage par renforcement pour l'optimisation en temps du transport des bagages par des véhicules guidés automatisés (AGV) en environnement aéroportuaire.

AUTEURE : Sayan Suos

Table des matières

Introduction	2
1 Installation du simulateur	3
1.1 Prérequis	3
1.2 Installation	3
2 Structure du projet	4
2.1 Organisation des fichiers	4
2.2 Architecture du simulateur	5
2.3 Organisation des classes	6
3 Configuration du simulateur	8
3.1 Configuration des expériences	8
3.2 Configuration des environnements	9
3.3 Configuration des agents	12
3.4 Configuration de la fonction de récompense	13
4 Utilisation du simulateur	15
4.1 Modes d'exécution	15
4.2 Workflow d'exécution	16
4.3 Entraînement d'une politique	16
4.4 Validation d'une politique	17
4.5 Évaluation d'une politique	17
5 Visualisation des résultats	19
5.1 Démonstration en temps réel	19
5.2 Génération d'une animation	19
5.3 Génération d'une figure comparative des scénarios	20
5.4 Résultats générés	21

Introduction

To do : Objectif du simulateur – Fonctionnalités principales – Architecture générale – Organisation du document

1 Installation du simulateur

Cette section décrit les différentes étapes nécessaires pour installer le simulateur sur une machine locale. Les procédures présentées ont été testées sous macOS et Windows.

1.1 Prérequis

Avant d'installer le simulateur, les logiciels suivants doivent être installés sur la machine :

- Python (version 3.11 ou supérieure) ;
- Git, utilisé pour cloner le dépôt du projet ;
- un environnement virtuel Python, permettant d'isoler les dépendances du projet des autres installations Python présentes sur la machine.

1.2 Installation

Le projet est distribué sous la forme d'un dépôt Git. La première étape consiste donc à cloner le dépôt sur la machine locale :

```
1 git clone https://github.com/sayansuos/baggage-handling-rl.git
2 cd baggage-handling-rl
```

Il est ensuite recommandé de créer un environnement virtuel afin d'isoler les dépendances du projet. Celui-ci peut ensuite être activé selon le système d'exploitation utilisé :

- Sous macOS ou Linux :

```
1 python -m venv .venv
2 source ./venv/bin/activate
```

- Sous Windows :

```
1 python -m venv .venv
2 .\venv\Scripts\activate
```

Une fois l'environnement virtuel activé, les dépendances du projet peuvent être installées à partir du fichier `requirements.txt` :

```
1 pip install -r requirements.txt
```

Une fois cette étape terminée, le simulateur est prêt à être utilisé.

2 Structure du projet

Cette section présente l'organisation générale du projet ainsi que l'architecture du simulateur. Elle décrit tout d'abord la structure des principaux fichiers et dossiers du dépôt, puis le fonctionnement global du simulateur au travers de son architecture fonctionnelle. Enfin, elle introduit les principales classes constituant le simulateur ainsi que leurs relations afin de faciliter la compréhension de son implémentation.

2.1 Organisation des fichiers

Le projet est organisé en plusieurs dossiers, chacun regroupant un ensemble de fonctionnalités spécifiques. Le rôle des principaux fichiers et dossiers est résumé dans le Tableau 1.

TABLE 1 – Description des principaux éléments du projet.

<i>Élément</i>	<i>Description</i>
<code>configs/</code>	Fichiers de configuration du simulateur, regroupant les paramètres des environnements, des agents, des fonctions de récompense et des expériences d'entraînement.
<code>docs/</code>	Documentation du projet, comprenant le manuel d'utilisation, le document de modélisation ainsi que les comptes rendus du projet.
<code>figures/</code>	Figures, graphiques et animations générés lors de l'entraînement et de l'évaluation du modèle.
<code>logs/</code>	Journaux d'exécution et métriques générés lors de l'entraînement et de l'évaluation du modèle.
<code>rl/</code>	Implémentation des algorithmes d'apprentissage par renforcement ainsi que des réseaux de neurones associés.
<code>simulator/</code>	Cœur du simulateur. Ce dossier regroupe les principaux composants de la simulation (environnement, agents, obstacles), les outils nécessaires à l'apprentissage par renforcement (observations, fonction de récompense), la gestion des interactions entre les entités ainsi que le calcul des trajectoires des obstacles dynamiques par A*.
<code>utils/</code>	Fonctions utilitaires utilisées par l'ensemble du projet (visualisation, journalisation, chargement des configurations, etc.).
<code>main.py</code>	Point d'entrée du projet, permettant de sélectionner le mode d'exécution du simulateur.
<code>runner.py</code>	Fonctions responsables du lancement des entraînements, des validations, des évaluations et de la génération des animations.

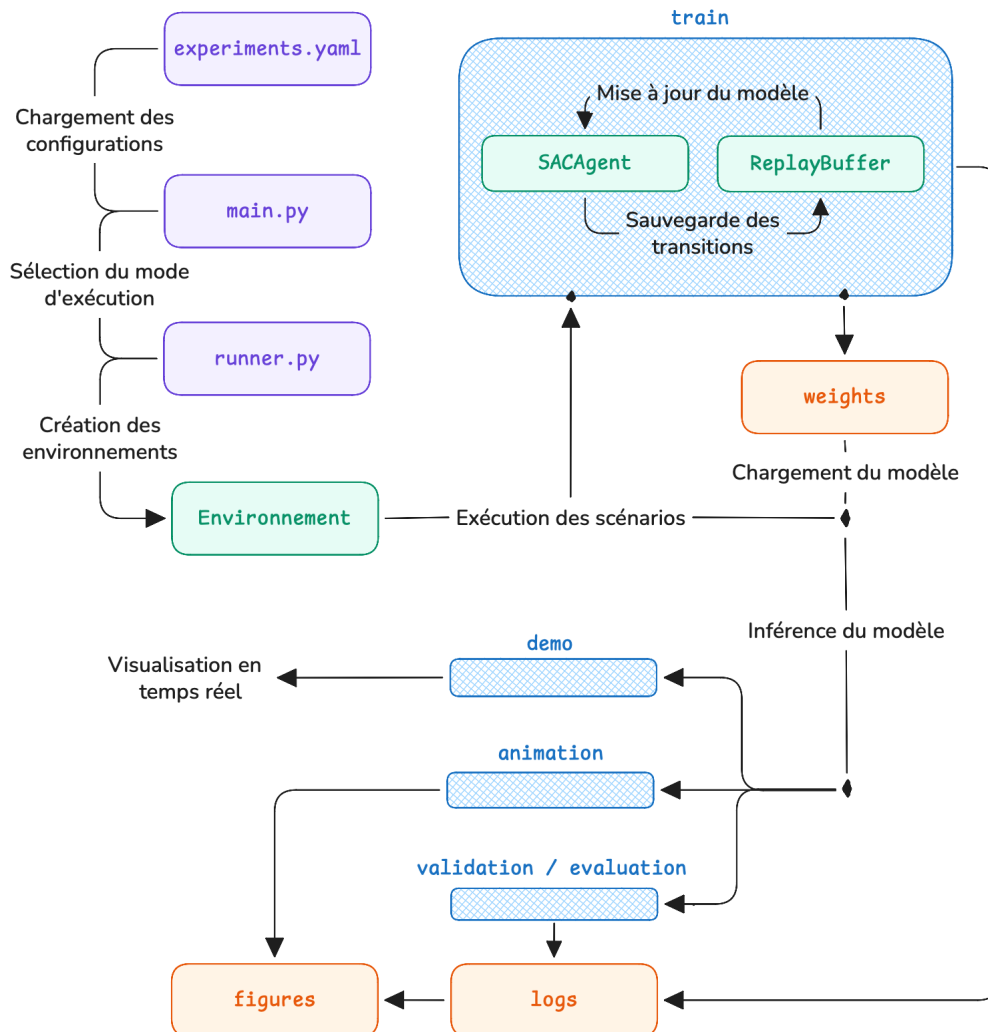
Remarque

Le dossier `simulator/` contient l'ensemble des composants nécessaires au fonctionnement du simulateur. Dans la plupart des cas, l'utilisateur n'a pas besoin de modifier ce dossier : la personnalisation du comportement du simulateur s'effectue principalement à travers les fichiers de configuration présentés dans la section suivante.

2.2 Architecture du simulateur

La Figure 1 présente l'architecture fonctionnelle du simulateur ainsi que les principales étapes d'exécution d'une expérience.

FIGURE 1 – Architecture fonctionnelle du simulateur



Quel que soit le mode d'exécution sélectionné (**train**, **validation**, **evaluation**, **animation** ou **demo**), le programme débute dans le fichier `main.py`. Celui-ci charge les fichiers de configuration afin d'initialiser l'environnement de simulation, les agents ainsi que les scénarios définis par l'utilisateur. Il sélectionne ensuite le mode d'exécution souhaité puis appelle les fonctions correspondantes du fichier `runner.py`. Une fois cette phase d'initialisation terminée, le comportement du simulateur dépend du mode d'exécution choisi dans le **runner**.

En mode **train**, un agent est entraîné à l'aide de l'algorithme Soft Actor-Critic (SAC). À chaque interaction avec l'environnement, les observations de l'environnement sont transmises à l'algorithme SAC, qui sélectionne une action ensuite appliquée à l'environnement. Cela produit un nouvel état et une récompense. Les transitions sont stockées dans le **Replay Buffer** et utilisées pour mettre à jour les réseaux de neurones. À la fin de l'entraînement, les poids du modèle ainsi que les différentes métriques sont enregistrés.

Les modes `validation`, `evaluation`, `animation` et `demo` utilisent quant à eux un modèle préalablement entraîné. Les poids du modèle préalablement entraîné sont chargés afin de calculer les actions de l'agent. Selon le mode sélectionné, le simulateur produit ensuite les métriques d'évaluation, les figures de résultats, une animation ou une visualisation en temps réel.

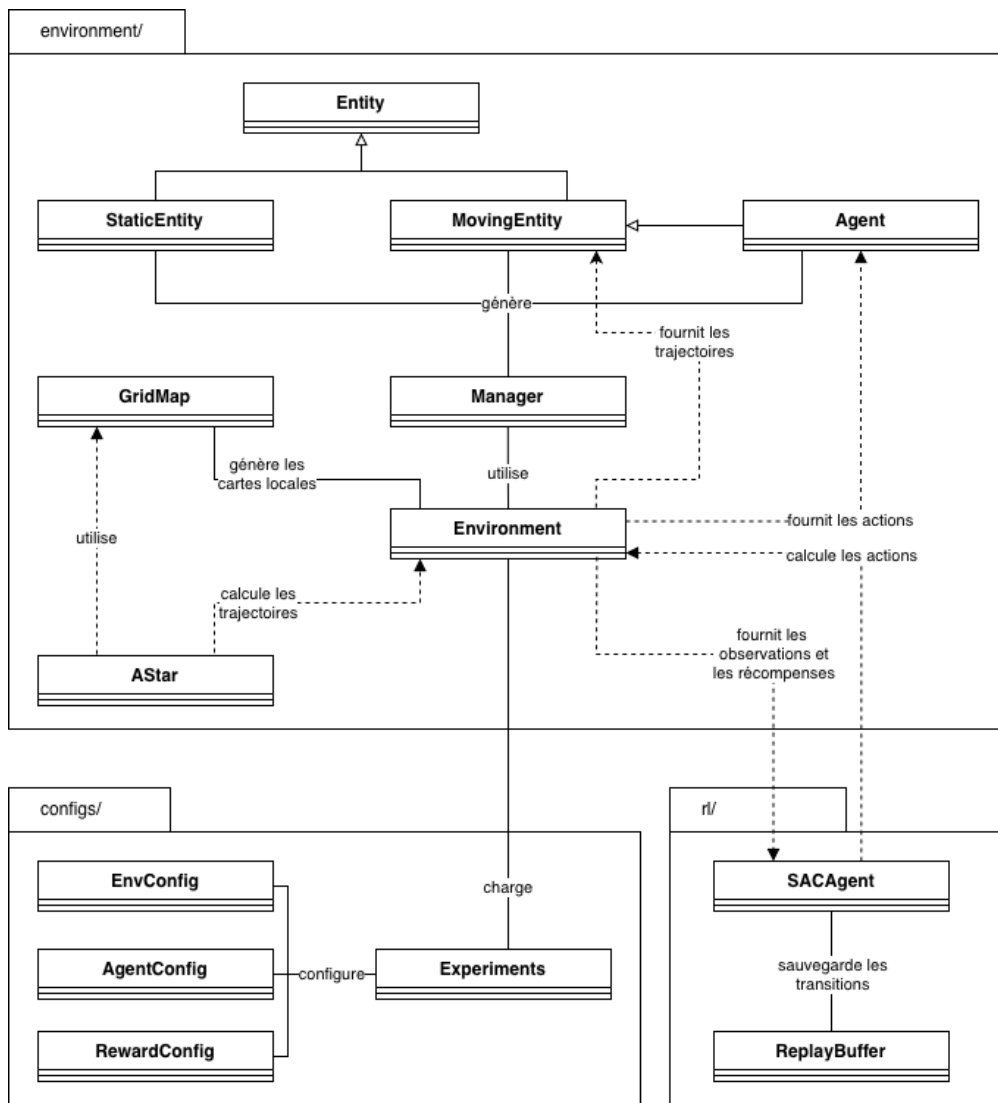
Remarque

Quel que soit le mode d'exécution choisi, la boucle principale de simulation reste identique. Seules les opérations réalisées autour de cette boucle (apprentissage, chargement des poids, génération de figures ou affichage en temps réel) diffèrent selon le mode sélectionné.

2.3 Organisation des classes

Le simulateur repose sur une architecture orientée objet dans laquelle les différents éléments présents dans un environnement sont représentés par des classes. La Figure 2 présente un diagramme de classes simplifié des principaux composants du simulateur.

FIGURE 2 – Diagramme de classes simplifié du simulateur.



Remarque

Le diagramme de classes présenté est volontairement simplifié. Les attributs et les méthodes des différentes classes n'y sont pas représentés afin de privilégier une vue d'ensemble de l'architecture du simulateur et des principales relations entre ses composants.

La classe `Entity` constitue la classe de base des objets présents dans l'environnement. Elle définit les propriétés communes aux différentes entités, notamment leur position, leurs dimensions ainsi que les mécanismes de détection des collisions.

Deux classes héritent de cette celle-ci : la classe `StaticEntity` qui représente les obstacles statiques de l'environnement, ainsi que la classe `MovingEntity` qui représente les entités capables de se déplacer, telles que des obstacles dynamiques. La classe `Agent` hérite de cette dernière et complète son comportement avec les propriétés nécessaires à la navigation et à l'apprentissage par renforcement.

La classe `Environment` regroupe les agents, les obstacles statiques et les obstacles dynamiques qui composent un scénario. Elle est responsable de leur initialisation (via la classe `Manager`), de l'application des actions et trajectoires, de la mise à jour de la simulation, du calcul des observations et des récompenses, ainsi que de la détection de la fin d'un épisode.

La classe `GridMap` fournit une représentation discrète de l'environnement. Elle est utilisée à la fois pour construire les cartes locales observées par les agents et pour calculer les trajectoires des obstacles dynamiques à l'aide de la classe `AStar`.

Enfin, l'apprentissage par renforcement est assuré par la classe `SACAgent`, qui s'appuie sur un `ReplayBuffer` ainsi que sur les réseaux de neurones `FeaturesExtractor`, `ActorNetwork` et `CriticNetwork`. Les paramètres des environnements, des agents et de la fonction de récompense sont quant à eux définis respectivement par les classes `EnvironmentConfig`, `AgentConfig` et `RewardConfig`.

3 Configuration du simulateur

La configuration du simulateur repose sur un ensemble de fichiers YAML regroupés dans le dossier `configs/`. Ces fichiers permettent de définir l'ensemble des paramètres nécessaires à l'exécution d'une expérience, notamment les caractéristiques des environnements, des agents, de la fonction de récompense ainsi que les expériences d'entraînement ou d'évaluation. Cette séparation entre la configuration et le code permet de modifier facilement le comportement du simulateur sans intervenir directement dans son implémentation.

TABLE 2 – Description des fichiers de configuration.

<i>Élément</i>	<i>Description</i>
<code>config.py</code>	Définit les différentes classes de configuration (dataclasses) utilisées pour représenter les paramètres des environnements, des agents, des fonctions de récompense et des expériences après le chargement des fichiers YAML.
<code>experiments.yaml</code>	Définit les expériences à exécuter.
<code>environments.yaml</code>	Définit les scénarios et les environnements.
<code>agent.yaml</code>	Définit les paramètres des agents.
<code>reward.yaml</code>	Définit les paramètres de la fonction de récompense.

Les fichiers de configuration sont automatiquement chargés par le module `config_loader` (dans le dossier `utils/`), qui lit les fichiers YAML et instancie les différentes classes de configuration (`EnvironmentConfig`, `AgentConfig`, `RewardConfig` et `Experiment`). L'utilisateur n'a donc qu'à modifier les fichiers de configuration sans intervenir dans le code source.

Les sections suivantes détaillent le contenu de chacun de ces fichiers de configuration.

3.1 Configuration des expériences

Les expériences à exécuter sont définies dans le fichier `experiments.yaml`. Une expérience correspond à un scénario d'entraînement ou d'évaluation et associe un environnement, une configuration d'agent ainsi qu'une fonction de récompense. Ce fichier permet également de définir le nombre d'étapes d'entraînement ou le nombre d'épisodes selon le mode d'exécution.

Chaque expérience d'entraînement est définie par une entrée de la liste `train_experiments`.

```
1 train_experiments:
2   - name: random_1
3     env_config: random_1
4     agent_config: agent_1
5     reward_config: reward_1
6     n_steps: 50_000
```

De même, chaque expérience d'évaluation est définie par une entrée de la liste `eval_experiments`.

```

1 eval_experiments:
2   - name: random_1
3     env_config: random_1
4     agent_config: agent_1
5     reward_config: reward_1
6     n_steps: null

```

TABLE 3 – Description des paramètres de Experiment.

Paramètres	Description
name	Nom de l'expérience. Ce nom est également utilisé pour identifier les fichiers de résultats et les poids sauvegardés.
env_config	Nom de la configuration d'environnement à utiliser (définie dans <code>environment.yaml</code>).
agent_config	Nom de la configuration de l'agent (définie dans <code>agent.yaml</code>).
reward_config	Nom de la fonction de récompense (définie dans <code>reward.yaml</code>).
n_steps	Nombre d'étapes d'entraînement. Ce paramètre est utilisé uniquement pour les expériences d'entraînement.

Plusieurs expériences peuvent être définies dans un même fichier. Elles sont exécutées dans l'ordre où elles apparaissent, ce qui permet notamment de mettre en place un curriculum d'apprentissage. Chaque expérience peut ainsi utiliser une configuration d'environnement différente afin d'augmenter progressivement la difficulté des scénarios, tandis que les poids du modèle appris lors d'une expérience sont réutilisés pour initialiser la suivante.

Remarque

Les expériences d'entraînement sont exécutées pendant un nombre d'étapes défini par le paramètre `n_steps`. Ce nombre doit être choisi en fonction de la difficulté du scénario ainsi que de la diversité des situations rencontrées durant l'apprentissage.

Les expériences d'évaluation n'utilisent pas ce paramètre. Elles sont exécutées sur un nombre fixe d'épisodes afin d'évaluer la politique préalablement entraînée. Ce nombre d'épisodes est défini dans le fichier `main.py`.

3.2 Configuration des environnements

Les environnements sont définis dans le fichier `environments.yaml`. Chaque environnement décrit un scénario de simulation en précisant les dimensions de la carte, les modes de génération des entités ainsi que les paramètres contrôlant la création des obstacles et le déroulement d'un épisode.

Chaque environnement est identifié par un nom unique, qui peut ensuite être référencé dans le fichier `experiments.yaml` à l'aide du paramètre `env_config`.

```

1  random_1:
2    width: 64
3    height: 32
4
5    agent_mode: random
6    env_mode: random
7
8    nb_agents: 1
9    nb_static_obstacles: 4
10   nb_moving_obstacles: 2
11   nb_targets: 2
12
13   margin: 3
14   max_attempts: 100
15   max_steps: 300
16
17   thickness: 1
18   radius_min: 0.5
19   radius_max: 1.0

```

TABLE 4 – Description des paramètres de `EnvironmentConfig`.

<i>Paramètre</i>	<i>Description</i>
<code>width</code>	Largeur de l’environnement.
<code>height</code>	Hauteur de l’environnement.
<code>agent_mode</code>	Définit le mode de génération des agents et de leurs objectifs.
<code>env_mode</code>	Définit le type d’environnement et la disposition des obstacles statiques.
<code>nb_agents</code>	Nombre d’agents présents dans le scénario.
<code>nb_static_obstacles</code>	Nombre d’obstacles statiques générés lorsque le mode d’environnement le permet.
<code>nb_moving_obstacles</code>	Nombre d’obstacles dynamiques générés.
<code>nb_targets</code>	Nombre de positions cibles successives attribuées à chaque agent ou obstacle dynamique.
<code>margin</code>	Distance minimale imposée entre les entités lors de leur génération.
<code>max_attempts</code>	Nombre maximal de tentatives pour générer une position valide avant l’abandon.
<code>max_steps</code>	Nombre maximal d’étapes autorisées pour un épisode.
<code>thickness</code>	Épaisseur des murs délimitant l’environnement.
<code>radius_min</code>	Rayon minimal des obstacles dynamiques générés aléatoirement.
<code>radius_max</code>	Rayon maximal des obstacles dynamiques générés aléatoirement.

Les paramètres `env_mode` et `agent_mode` déterminent la stratégie de génération des scénarios. Selon les valeurs choisies, les positions des agents, des obstacles statiques, des obstacles dynamiques et

des cibles sont générées suivant des règles différentes, permettant de construire des environnements adaptés à différents niveaux de difficulté. Le code source de ces configurations se trouvent dans la classe `Manager`.

TABLE 5 – Description des modes de génération.

<i>Paramètre</i>	<i>Valeur</i>	<i>Description</i>
env_mode	random	Génère aléatoirement les positions et les dimensions des obstacles statiques dans l’environnement.
	warehouse	Génère une disposition prédéfinie d’obstacles statiques représentant des allées d’entrepôt.
	crossing	Génère les obstacles statiques autour du centre de l’environnement selon une disposition circulaire.
	fixed_n	Génère les obstacles statiques sur n colonnes fixes, avec des dimensions déterministes.
	fixed_random_n	Génère les obstacles statiques sur n colonnes fixes, avec des dimensions aléatoires.
agent_mode	random	Génère aléatoirement les positions initiales et les cibles des agents et des obstacles dynamiques, en respectant les contraintes de distance minimale.
	fixed_n	Génère les positions initiales et les cibles des agents de part et d’autre des n colonnes d’obstacles statiques. Les obstacles dynamiques sont placés dans la partie supérieure ou inférieure de l’environnement, avec des cibles situées dans la zone opposée.
	crossing	Génère les positions initiales des agents et des obstacles dynamiques sur un cercle, puis leur attribue une cible diamétralement opposée afin de créer un scénario de croisement.

Selon le mode de génération sélectionné, certains paramètres de la configuration peuvent être ignorés. Par exemple, le mode `warehouse` repose sur une disposition prédéfinie des obstacles statiques et ne permet donc pas de contrôler directement leur nombre ou leurs dimensions. De même, dans les modes `fixed_n`, `fixed_random_n` et `crossing`, les dimensions des obstacles sont déterminées automatiquement à partir des dimensions de l’environnement et du nombre d’obstacles demandé. Pour le mode `random`, les dimensions des obstacles sont également adaptées afin de respecter les contraintes géométriques imposées, notamment les distances minimales entre les différentes entités.

Remarque

Les différents modes de génération ont été conçus pour produire des scénarios favorables à l'apprentissage par renforcement. Les positions des agents, des obstacles et des cibles ne sont donc pas uniquement générées de manière aléatoire, mais suivent des règles permettant de reproduire les situations recherchées (croisement, franchissement d'obstacles, etc.). Cette méthode favorise des interactions pertinentes entre les agents et leur environnement, tout en garantissant une diversité suffisante des scénarios rencontrés durant l'apprentissage.

3.3 Configuration des agents

Les caractéristiques des agents sont définies dans le fichier `agent.yaml`. Cette configuration regroupe les paramètres physiques du robot ainsi que les paramètres définissant son espace des observations et son espace des actions.

Chaque configuration d'agent est identifiée par un nom unique, qui peut ensuite être référencé dans le fichier `experiments.yaml` à l'aide du paramètre `agent_config`.

```
1 agent_1:
2   n_maps: 3
3
4   v_min: 0.0
5   v_max: 5.0
6
7   omega_min: -1.047
8   omega_max: 1.047
9
10  radius: 0.5
11  length_view: 11
```

TABLE 6 – Description des paramètres de `AgentConfig`.

Paramètre	Description
<code>n_maps</code>	Nombre de cartes locales successives conservées dans l'observation afin d'intégrer une information temporelle.
<code>v_min</code>	Vitesse linéaire minimale autorisée pour l'agent.
<code>v_max</code>	Vitesse linéaire maximale autorisée pour l'agent.
<code>omega_min</code>	Vitesse angulaire minimale autorisée pour l'agent.
<code>omega_max</code>	Vitesse angulaire maximale autorisée pour l'agent.
<code>radius</code>	Rayon de l'agent utilisé pour la détection des collisions.
<code>length_view</code>	Taille de la carte locale observée par l'agent.

Les paramètres `v_min`, `v_max`, `omega_min` et `omega_max` définissent directement les bornes de l'espace des actions de l'agent. Les commandes calculées par l'algorithme d'apprentissage sont

automatiquement ramenées dans ces intervalles avant d’être appliquées au simulateur.

Remarque

Les paramètres `n_maps` et `length_view` contrôlent la quantité d’informations spatiales et temporelles disponible pour l’agent. Une carte locale plus grande ou un nombre plus important de cartes successives permettent de représenter plus précisément l’environnement et l’évolution des obstacles, mais augmentent également la dimension des observations traitées par l’algorithme d’apprentissage.

3.4 Configuration de la fonction de récompense

Les paramètres de la fonction de récompense sont définis dans le fichier `reward.yaml`. Cette configuration permet d’ajuster l’importance des différentes composantes de la récompense utilisées durant l’apprentissage, telles que la progression vers la cible, l’évitement des obstacles ou la limitation des rotations inutiles.

Chaque configuration de récompense est identifiée par un nom unique, qui peut ensuite être référencé dans le fichier `experiments.yaml` à l’aide du paramètre `reward_config`.

```
1 reward_1:
2   beta1: 5.0
3   beta2: 0.1
4   beta3: 0.5
5
6   goal_bonus: 400.0
7   collision_malus: -100.0
8
9   angular_malus: -2.0
10
11   safety_malus: -1.0
12   safety_threshold: 2.0
```

TABLE 7 – Description des paramètres de `RewardConfig`.

Paramètre	Description
<code>beta1</code>	Pondération du terme de récompense lié à la progression vers la cible.
<code>beta2</code>	Pondération du terme de récompense lié au respect des distances de sécurité.
<code>beta3</code>	Pondération du terme de récompense lié à la rotation de l’agent.
<code>goal_bonus</code>	Récompense attribuée lorsque l’agent atteint sa cible.
<code>collision_malus</code>	Pénalité appliquée lorsqu’une collision est détectée.

<i>Paramètre</i>	<i>Description</i>
angular_malus	Pénalité appliquée en fonction de la vitesse angulaire de l'agent.
safety_malus	Pénalité appliquée lorsque l'agent évolue à une distance inférieure au seuil de sécurité.
safety_threshold	Distance de sécurité à partir de laquelle la pénalité de sécurité est appliquée.

Les paramètres de cette configuration permettent d'ajuster l'importance relative des différentes composantes de la récompense. En modifiant les coefficients de pondération ou les bonus et pénalités associés à certains comportements, il est possible d'orienter l'apprentissage vers des politiques privilégiant, par exemple, une navigation plus rapide, plus prudente ou plus fluide.

La formulation mathématique complète de la fonction de récompense ainsi que le rôle de chacune de ses composantes sont présentés dans le document de modélisation.

Remarque

La fonction de récompense influence directement le comportement appris par l'agent. Une modification importante de ses paramètres modifie l'objectif d'optimisation de l'algorithme d'apprentissage et nécessite un nouvel entraînement complet du modèle.

4 Utilisation du simulateur

Cette section décrit les principaux modes d'exécution du simulateur ainsi que la procédure permettant d'entraîner, de valider et d'évaluer une politique d'apprentissage par renforcement. Elle présente également le déroulement général d'une expérience, depuis son lancement jusqu'à la production des résultats.

Les modes dédiés à la visualisation des résultats, tels que la démonstration en temps réel, la génération d'animations ou la représentation des scénarios, sont quant à eux décrits dans la section 5.

4.1 Modes d'exécution

Le simulateur propose plusieurs modes d'exécution permettant d'entraîner un agent, d'évaluer une politique existante ou encore de générer des visualisations des résultats. Le mode d'exécution est sélectionné dans le fichier `main.py` avant le lancement du programme.

TABLE 8 – Description des modes d'exécution.

<i>Mode</i>	<i>Description</i>
<code>train</code>	Entraîne successivement un agent SAC sur l'ensemble des expériences du curriculum d'entraînement. À la fin de chaque expérience, les poids du modèle sont sauvegardés avant d'être utilisés comme point de départ de l'expérience suivante. Les métriques et les figures associées sont ensuite sauvegardées afin d'analyser les performances de l'entraînement. Fait appel aux fonctions <code>run_train</code> et <code>run_validation</code> .
<code>validation</code>	Charge chaque politique entraînée et l'évalue uniquement sur le scénario correspondant à son entraînement. Les métriques et les figures associées sont ensuite sauvegardées afin d'analyser les performances de la politique sur son scénario d'apprentissage. Fait appel à la fonction <code>run_validation</code> .
<code>evaluation</code>	Charge une politique entraînée et l'évalue sur l'ensemble des scénarios du curriculum d'évaluation. Ce mode permet d'étudier la capacité de généralisation des politiques apprises. Les métriques et les figures correspondantes sont sauvegardées. Fait appel à la fonction <code>run_evaluation</code> .
<code>demo</code>	Charge une politique entraînée et affiche une visualisation en temps réel d'une exécution pour chaque expérience des curriculums d'entraînement et d'évaluation. Fait appel à la fonction <code>run_demo</code> .
<code>animation</code>	Charge une politique entraînée et génère, pour chaque expérience des curriculums d'entraînement et d'évaluation, une animation de l'exécution (GIF et MP4) ainsi qu'une représentation du scénario sous forme de grille. Fait appel aux fonctions <code>plot_renders</code> et <code>run_animation</code> .

4.2 Workflow d'exécution

Après avoir configuré les expériences et sélectionné le mode d'exécution souhaité dans le fichier `main.py`, le simulateur est lancé depuis la racine du projet. Selon le mode choisi, il entraîne une nouvelle politique ou charge une politique préalablement entraînée afin de produire les métriques et les figures correspondantes.

Le lancement du simulateur s'effectue simplement à l'aide de la commande suivante :

```
1 python main.py
```

La Figure 3 résume le déroulement classique d'une expérience, depuis la configuration des scénarios jusqu'à l'exploitation des résultats.

FIGURE 3 – Schéma du workflow d'exécution

4.3 Entraînement d'une politique

La fonction `run_train` entraîne successivement une politique sur les expériences du curriculum d'entraînement. À la fin de chaque expérience, les poids du modèle, les métriques d'apprentissage et les figures associées sont sauvegardés.

```
1 run_train(  
2     experiments=train_experiments,  
3     previous_exp=None,  
4     trained_agent_id="agent_1",  
5 )
```

TABLE 9 – Paramètres de la fonction `run_train`.

Paramètre	Par défaut	Description
<code>experiments</code>	-	Liste des expériences constituant le curriculum d'entraînement.
<code>previous_exp</code>	<code>None</code>	Expérience dont les poids doivent être chargés avant le début de l'entraînement. Lorsque la valeur est <code>None</code> , la première politique est initialisée sans poids provenant d'une expérience précédente.
<code>trained_agent_id</code>	<code>agent_1</code>	Identifiant de l'agent dont la politique doit être entraînée.

Lorsque plusieurs expériences sont fournies, elles sont exécutées dans l'ordre de la liste. Les poids obtenus à la fin d'une expérience sont alors utilisés pour initialiser la suivante, ce qui permet de mettre en œuvre un curriculum d'apprentissage.

Remarque

Le paramètre `trained_agent_id` doit correspondre à l'identifiant de l'agent utilisé lors de l'entraînement de la politique. Si plusieurs agents ont été entraînés indépendamment, cet identifiant permet de charger les poids associés à l'agent souhaité.

4.4 Validation d'une politique

La fonction `run_validation` évalue chaque politique sur le scénario correspondant à celui sur lequel elle a été entraînée. Elle sauvegarde les métriques, les figures et, lorsque cela est demandé, plusieurs épisodes sous forme d'animation.

```
1 run_validation(  
2     experiments=train_experiments,  
3     n_episodes=100,  
4     n_render=5,  
5     trained_agent_id="agent_1",  
6 )
```

TABLE 10 – Paramètres de la fonction `run_validation`.

Paramètre	Par défaut	Description
<code>experiments</code>	-	Liste des expériences d'entraînement à valider. Chaque politique est évaluée sur le scénario correspondant à son apprentissage.
<code>n_episodes</code>	100	Nombre d'épisodes exécutés pour estimer les performances de chaque politique.
<code>n_render</code>	5	Nombre d'épisodes dont les images sont conservées afin de produire une animation. Une valeur égale à 0 désactive la génération des animations.
<code>trained_agent_id</code>	agent_1	Identifiant de l'agent contrôlé par la politique évaluée.

Remarque

Contrairement à l'entraînement, les modes `validation` et `evaluation` utilisent une politique déterministe. Les actions sont donc directement calculées à partir de la moyenne de la distribution apprise, sans échantillonnage aléatoire, afin d'obtenir un comportement stable et reproductible.

4.5 Évaluation d'une politique

La fonction `run_evaluation` évalue une politique donnée sur l'ensemble des scénarios du curriculum d'évaluation. Contrairement à la validation, la même politique est donc appliquée à plusieurs

environnements afin d'étudier sa capacité de généralisation.

```
1 run_evaluation(  
2     experiments=eval_experiments,  
3     policy=train_experiments[-1],  
4     n_episodes=100,  
5     n_render=5,  
6     trained_agent_id="agent_1",  
7 )
```

TABLE 11 – Paramètres de la fonction `run_evaluation`.

<i>Paramètre</i>	<i>Par défaut</i>	<i>Description</i>
<code>experiments</code>	-	Liste des scénarios sur lesquels la politique doit être évaluée. Il s'agit généralement de la liste <code>eval_experiments</code> .
<code>policy</code>	-	Expérience d'entraînement identifiant les poids de la politique à charger.
<code>n_episodes</code>	100	Nombre d'épisodes exécutés pour chaque scénario d'évaluation.
<code>n_render</code>	5	Nombre d'épisodes enregistrés sous forme d'animation pour chaque scénario. Une valeur égale à 0 désactive cette génération.
<code>trained_agent_id</code>	<code>agent_1</code>	Identifiant de l'agent contrôlé par la politique évaluée.

5 Visualisation des résultats

Cette section présente les différents outils permettant de visualiser les résultats produits par le simulateur. Elle décrit les fonctions permettant d’observer l’exécution d’une politique en temps réel, de générer des animations, de représenter graphiquement les scénarios ainsi que les principaux fichiers générés au cours des différentes phases d’exécution.

5.1 Démonstration en temps réel

La fonction `run_demo` exécute un épisode pour chacune des expériences fournies. Lorsque le rendu est activé, l’évolution des agents et des obstacles est affichée en temps réel.

```
1 run_demo(  
2     experiments=eval_experiments,  
3     policy=train_experiments[-1],  
4     render=True,  
5     trained_agent_id="agent_1",  
6 )
```

TABLE 12 – Paramètres de la fonction `run_demo`.

<i>Paramètre</i>	<i>Par défaut</i>	<i>Description</i>
<code>experiments</code>	-	Liste des scénarios à exécuter. Un épisode est lancé pour chaque expérience de la liste.
<code>policy</code>	-	Expérience d’entraînement identifiant les poids de la politique à charger.
<code>render</code>	<code>False</code>	Active ou désactive le rendu graphique en temps réel. Lorsque la valeur est <code>False</code> , les épisodes sont exécutés sans affichage.
<code>trained_agent_id</code>	<code>agent_1</code>	Identifiant de l’agent contrôlé par la politique chargée.

Les actions sont calculées de manière déterministe durant la démonstration afin d’obtenir un comportement stable et reproductible de la politique entraînée.

5.2 Génération d’une animation

La fonction `run_animation` exécute un épisode pour chaque scénario, enregistre les images successives du rendu puis les rassemble dans une animation.

```

1 run_animation(
2     experiments=eval_experiments,
3     policy=train_experiments[-1],
4     path="figures/animations",
5     file_name="evaluation",
6     fps=20,
7     trained_agent_id="agent_1",
8 )

```

TABLE 13 – Paramètres de la fonction `run_animation`.

<i>Paramètre</i>	<i>Valeur par défaut</i>	<i>Description</i>
<code>experiments</code>	-	Liste des scénarios à exécuter et à inclure dans l'animation.
<code>policy</code>	-	Expérience d'entraînement identifiant les poids de la politique à charger.
<code>path</code>	-	Dossier dans lequel les fichiers d'animation doivent être enregistrés.
<code>file_name</code>	-	Nom utilisé pour les fichiers générés.
<code>fps</code>	20	Nombre d'images affichées par seconde dans l'animation produite.
<code>trained_agent_id</code>	agent_1	Identifiant de l'agent contrôlé par la politique.

5.3 Génération d'une figure comparative des scénarios

La fonction `plot_renders` génère une représentation statique de l'état initial de chaque expérience fournie. Les différents scénarios sont regroupés dans une même figure afin de faciliter leur comparaison visuelle. Pour chaque expérience, un environnement est créé puis réinitialisé avec une graine fixe avant d'être affiché dans une sous-figure portant le nom de l'expérience.

```

1 plot_renders(
2     experiments=eval_experiments,
3     path="figures/renders",
4     file_name="evaluation",
5     max_ncols=2,
6 )

```

Remarque

La fonction utilise une graine fixe égale à 1234 lors de la réinitialisation des environnements. Elle permet ainsi de générer des représentations reproductibles des scénarios et de comparer leur organisation dans des conditions identiques.

TABLE 14 – Paramètres de la fonction `plot_renders`.

<i>Paramètre</i>	<i>Par défaut</i>	<i>Description</i>
<code>experiments</code>	-	Liste des expériences dont les environnements doivent être représentés. Une sous-figure est générée pour chaque expérience.
<code>path</code>	-	Dossier dans lequel la figure comparative doit être enregistrée. pas.
<code>file_name</code>	""	Préfixe utilisé pour nommer le fichier généré. La figure est enregistrée sous la forme <code><file_name>_renders.png</code> .
<code>max_ncols</code>	2	Nombre maximal de colonnes utilisé pour organiser les sous-figures.

5.4 Résultats générés

Références

Table des figures

1	Architecture fonctionnelle du simulateur	5
2	Diagramme de classes simplifié du simulateur.	6
3	Schéma du workflow d'exécution	16

Liste des tableaux

1	Description des principaux éléments du projet.	4
2	Description des fichiers de configuration.	8
3	Description des paramètres de <code>Experiment</code>	9
4	Description des paramètres de <code>EnvironmentConfig</code>	10
5	Description des modes de génération.	11
6	Description des paramètres de <code>AgentConfig</code>	12
7	Description des paramètres de <code>RewardConfig</code>	13
8	Description des modes d'exécution.	15
9	Paramètres de la fonction <code>run_train</code>	16
10	Paramètres de la fonction <code>run_validation</code>	17
11	Paramètres de la fonction <code>run_evaluation</code>	18
12	Paramètres de la fonction <code>run_demo</code>	19
13	Paramètres de la fonction <code>run_animation</code>	20
14	Paramètres de la fonction <code>plot_renders</code>	21